

UNIVERSIDADE FEDERAL DE ITAJUBÁ
CAMPUS ITAJUBÁ

ENZO KOZONOE

RAFAEL CARDOSO DE AZEVEDO

STEVAN MACIEL RIBEIRO DE SOUZA

TRABALHO DE COMPILADORES:
DOCUMENTAÇÃO DA LINGUAGEM SER

ITAJUBÁ, MG

2024

INSTITUTO DE ENGENHRIA DE SISTEMAS E TECNOLOGIA DA INFORMAÇÃO
COMPILADORES – ECOM06A

ENZO KOZONOE

RAFAEL CARDOSO DE AZEVEDO

STEVAN MACIEL RIBEIRO DE SOUZA

TRABALHO DE COMPILADORES:
DOCUMENTAÇÃO DA LINGUAGEM SER

Relatório apresentado ao curso de
graduação em Engenharia de
Computação na Universidade Federal
de Itajubá, como requisito para nota N1.

Docente: Thatyana de Faria Piola
Seraphim

ITAJUBÁ, MG

2024

SUMÁRIO

1. EXPRESSÕES REGULARES.....	4
1.1. Operadores Relacionais.....	6
1.2. Operadores Lógicos.....	6
1.3. Operadores Aritméticos.....	7
1.4. Símbolos Especiais.....	8
1.5. Bloco de Comando.....	9
2. TOKENS	9
3. PALAVRAS RESERVADAS.....	11
4. AUTÔMATOS.....	11

1. EXPRESSÕES REGULARES

Início do programa: “commence”

- Simplesmente corresponde à palavra reservada “commence”, que é usada para indicar o início de um programa.

Fim do programa: “terminate”

- Similar à anterior, corresponde à palavra reservada “terminate”, que indica o fim do programa.

Letra: [a-z] | [A-Z]

- Padrão que corresponde a qualquer letra minúscula ou maiúscula do alfabeto.

Dígito: [0-9]

- Corresponde a qualquer dígito numérico de 0 a 9.

Variável: (letra) ((dígito) | (letra))*

- Uma letra seguida opcionalmente por uma sequência de dígitos ou de letras, indicando o formato de uma variável.

Inteiro: (“ - ”) ? (dígito) (dígito)*

- O número inteiro pode começar com sinal negativo (“ - ”) ou um dígito, seguido por zero ou mais dígitos.

Real: (“ - ”) ? (dígito) (dígito)* (separador) (dígito)+

- O número real pode começar com um sinal negativo (“ - ”) ou um dígito, seguido por zero ou mais dígitos, seguido por um separador (“ , ” ou “ . ”) e pelo menos um dígito após.

Número: inteiro | real

- Corresponde a qualquer número inteiro ou real.

Caractere: letra | dígito

- Corresponde a qualquer letra ou dígito individual.

Argumento: número | caractere

- O argumento corresponde a qualquer número ou caractere individual.

Operação: (argumento | variável) ((aritmético) (argumento | variável))+

- A operação pode ser um argumento ou uma variável, seguido por pelo menos uma sequência de operador aritmético juntamente com argumento ou variável.

Atribuição: (variável) (“ “) * (“ = ”) (“ ”) * ((argumento) | (operação))

- Na instrução de atribuição, uma variável recebe um argumento ou uma operação.

Entrada: (“input”) (abre chaves) (variável) (fecha chaves)

- A palavra reservada “input” indica uma entrada de dados no programa, que armazena o valor em uma variável dada entre chaves.

Saída: (“print”) (abre chaves) (“ ”) (argumento | variável | espaço) * (“ “ ”) (fecha chaves)

- A palavra reservada “print” indica uma saída de dados do programa, seguida por uma sequência de argumentos ou variáveis colocados entre aspas que, por sua vez, estão entre chaves.

Condição: ((not)? (operação) | (argumento) | (variável)) (relacional) ((operação) | (argumento) | (variável)) (((and | or) (not)? (operação) | (argumento) | (variável)) (relacional) ((operação) | (argumento) | (variável))))*

- A condição lógica pode envolver operações, argumentos ou variáveis relacionados por um operador relacional, que pode estar relacionada a uma outra condição através de um operador lógico.

If: (“if”) (“ “) * (abre parênteses) (condição) (fecha parênteses) (início do bloco) (comando) + (fim do bloco) (else)?

- A estrutura condicional “if” começa obrigatoriamente com a palavra reservada “if” seguida por uma condição entre parênteses, seguida por um bloco de comandos. O “else” é opcional.

Else: (“ else ”) (if)? ((início do bloco) (comando) + (fim do bloco)

- A estrutura condicional “else” começa obrigatoriamente com a palavra reservada “else” que pode estar seguida por um novo “if” ou por um bloco de comandos.

For: (“for”) (“ “) * (abre parênteses) (atribuição) (ponto e vírgula) (condição)(ponto e vírgula) ((variável (decrementa | incrementa)) | (operação)) (fecha parênteses) (início do bloco) (comando)+ (fim do bloco)

- A estrutura de repetição “for” começa obrigatoriamente com a palavra reservada “for”, seguida por uma inicialização (por meio de uma atribuição), uma condição e uma iteração (variável incrementada ou decrementada, ou uma operação), tudo entre parênteses, seguido por um bloco de comandos.

While: (“while”) (abre parênteses) (condição) (fecha parênteses) (início do bloco) (comando)+ (fim do bloco)

- A estrutura de repetição “while” começa obrigatoriamente com a palavra reservada “while”, seguida por uma condição entre parênteses, seguido por um bloco de comandos.

Comando: (entrada | saída | operação | atribuição | if | (if)(else) | for | while)(ponto e vírgula)

- Qualquer comando válido no programa, seguido por um ponto e vírgula. Isso inclui entrada, saída, operação, atribuição, estrutura condicionais “if” e “else”, e estruturas de repetição “for” e “while”.

1.1. Operadores Relacionais

Recebe: “=”

- Operador de atribuição, representado pela igualdade “=”.

Igual: “==”

- Representa a igualdade entre dois valores, indicada por “==”.

Maior ou igual: “>=”

- Indica que um valor é maior ou igual a outro, representado por “>=”.

Menor ou igual: “<=”

- Indica que um valor é menor ou igual a outro, representado por “<=”.

Maior: “>”

- Representa a comparação de que um valor é estritamente maior que outro, indicado por “>”.

Menor: “<”

- Representa a comparação de que um valor é estritamente menor que outro, indicado por “<”.

Diferente: “!=”

- Indica que dois valores são diferentes um do outro, representado por “!=”.

Relacional: “==” | “>=” | “<=” | “>” | “<” | “!=”

- Qualquer um dos operadores definidos anteriormente pode ser usado em uma expressão relacional.

1.2. Operadores Lógicos

And: (“and” | “&”)

- Representa a operação lógica “E”, na qual as duas condições devem ser verdadeiras para que a expressão como um todo seja verdadeira. Pode ser representada textualmente pela palavra reservada “and” ou pelo símbolo “&”.

Or: (“or” | “|”)

- Representa a operação lógica “OU”, na qual pelo menos uma das condições deve ser verdadeira para que a expressão como um todo seja verdadeira. Pode ser representada textualmente pela palavra reservada “or” ou pelo símbolo “|”.

Not: (“not” | “!”)

- Representa a operação lógica “NÃO”, que inverte o valor de uma condição, ou seja, caso a condição seja verdadeira, “not” a torna falsa e vice-versa. Pode ser representada textualmente pela palavra reservada “not” ou pelo símbolo “!”.

Lógico: (“and” | “&”) | (“or” | “|”) | (“not” | “!”)

- Qualquer um dos operadores definidos anteriormente pode ser usado em uma expressão lógica.

1.3. Operadores Aritméticos

Soma: “+”

- Representa a operação de adição, indicada pelo símbolo “+”.

Subtração: “-”

- Representa a operação de subtração, indicada pelo símbolo “-”.

Multiplicação: “*”

- Representa a operação de multiplicação, indicada pelo símbolo “*”.

Divisão: “/”

- Representa a operação de divisão, indicada pelo símbolo “/”.

Mod: “%”

- Representa a operação de módulo, que retorna o resto da divisão entre dois números inteiros, indicada pelo símbolo “%”.

Incrementa: “++”

- Representa a operação de incremento, no qual o valor de uma variável é aumentado em uma unidade, indicada pelo símbolo “++”.

Decrementa: "--"

- Representa a operação de decremento, no qual o valor de uma variável é diminuído em uma unidade, indicada pelo símbolo "--".

Aritmético: "+" | "-" | "*" | "/" | "%"

- Qualquer um dos operadores definidos para 2 argumentos anteriormente pode ser usado em uma expressão aritmética.

1.4. Símbolos Especiais

Separador: "," | "."

- Define os símbolos que separam elementos, pode ser uma vírgula "," ou um ponto ".".

Espaço: " "

- Define o símbolo de espaçamento entre elementos " ".

Dois pontos: ":"

- Representa o símbolo de dois pontos ":".

Ponto e vírgula: ";"

- Define o símbolo de ponto e vírgula ";", usado para separar comandos.

Abre parênteses: "("

- Representa o símbolo de abertura de parênteses "(", usado para indicar o início de uma expressão.

Fecha parênteses: ")"

- Representa o símbolo de fechamento de parênteses ")", usado para indicar o fim de uma expressão que começou com um parêntese aberto.

Abre chaves: "{"

- Representa o símbolo de abertura de chaves "{", usado para indicar o início de uma expressão.

Fecha chaves: "}"

- Representa o símbolo de fechamento de chaves "}", usado para indicar o fim de uma expressão que começou com uma chave aberta.

1.5. Bloco de Comando

Início do bloco: “begin”

- Palavra reservada “begin” que marca o início de um bloco de código, geralmente, usado para delimitar estrutura de controles, como repetições e condicionais.

Fim do bloco: “end”

- Palavra reservada “end” que marca o fim de um bloco de código, geralmente, usado para delimitar estrutura de controles, como repetições e condicionais, e que foi aberto anteriormente por “begin”.

2. TOKENS

Tabela 1 - Tokens

DEFINIÇÃO	TOKEN
Marca o início do programa	INICIO
Marca o fim do programa	FIM
Caractere do alfabeto	LETRA
Caractere numérico	DIGITO
Declaração de uma variável	VAR
Tipo de dado inteiro	INT
Tipo de dado real	REAL
Dado inteiro ou real	NUMERO
Tipo de dado caractere	CHAR
Entrada de dados	INPUT
Saída de dados	PRINT
Valor passado para um procedimento	ARGUMENTO
Inicia uma estrutura condicional	IF
Estrutura a ser executada se a condição IF não for atendida	ELSE
Estrutura de repetição para um número fixo de iterações	FOR
Estrutura de repetição enquanto uma condição é verdadeira	WHILE
Operação aritmética	OPERACAO
Avaliar se um ou mais condições são verdadeiras	CONDICAO
Instrução que executa uma ação específica	COMANDO
Atribui um valor a uma variável	ATRIBUICAO
Operador de atribuição	RECEBE
Compara dois valores	RELACIONAL

Operador que compara a igualdade entre dois valores	IGUAL
Operador que indica se um valor é maior ou igual a outro	MAIORIGUAL
Operador que indica se um valor é maior a outro	MAIOR
Operador que indica se um valor é menor ou igual a outro	MENORIGUAL
Operador que indica se um valor é menor a outro	MENOR
Operador que indica se dois valores são diferentes um do outro	DIFERENTE
Operador lógico que retorna verdadeiro se ambas as condições forem verdadeiras	AND
Operador lógico que retorna verdadeiro se pelo menos uma das condições for verdadeira	OR
Operador lógico que inverte o valor lógico da condição	NOT
Qualquer operador lógico	LOGICO
Operador de adição	SOMA
Operador de subtração	SUBTRACAO
Operador de multiplicação	MULTIPLICACAO
Operador de divisão	DIVISAO
Operador de módulo, que retorna o resto da divisão entre dois números inteiros	MOD
Operador de incremento, no qual o valor de uma variável é aumentado em uma unidade	INCREMENTA
Operador de decremento, no qual o valor de uma variável é diminuído em uma unidade	DECREMENTA
Qualquer operador aritmético	ARITMETICO
Utilizado para separar elementos	SEPARADOR
Utilizado para representar espaçamento entre elementos	ESPAÇO
Caractere para indicar o começo do que aparecerá na saída	ABREASPAS
Caractere para indicar o fim do que aparecerá na saída	FECHAASPAS
Dois pontos	DOISPONTOS
Caractere para indicar fim de uma instrução	PONTOVIRGULA
Caractere que indica um início de uma expressão	ABREPARENTESES
Caractere que indica um fim de uma expressão	FECHAPARENTESES

Caractere que indica um início de uma chamada de função	ABRECHAVES
Caractere que indica um fim de uma chamada de função	FECHACHAVES
Marca o início de um bloco de código	INICIOBLOCO
Marca o fim de um bloco de código	FIMBLOCO

3. PALAVRAS RESERVADAS

Tabela 2 - Palavras reservadas

commence
terminate
int
real
char
input
print
if
else
for
while
and
or
not
begin
end

4. AUTÔMATOS

- Variável;



Figura 1 - Autômato da expressão regular Variável

- Inteiro;

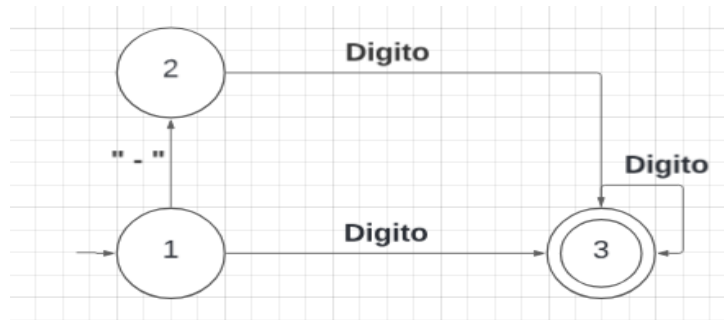


Figura 2 - Autômato da expressão regular Inteiro

- Real;

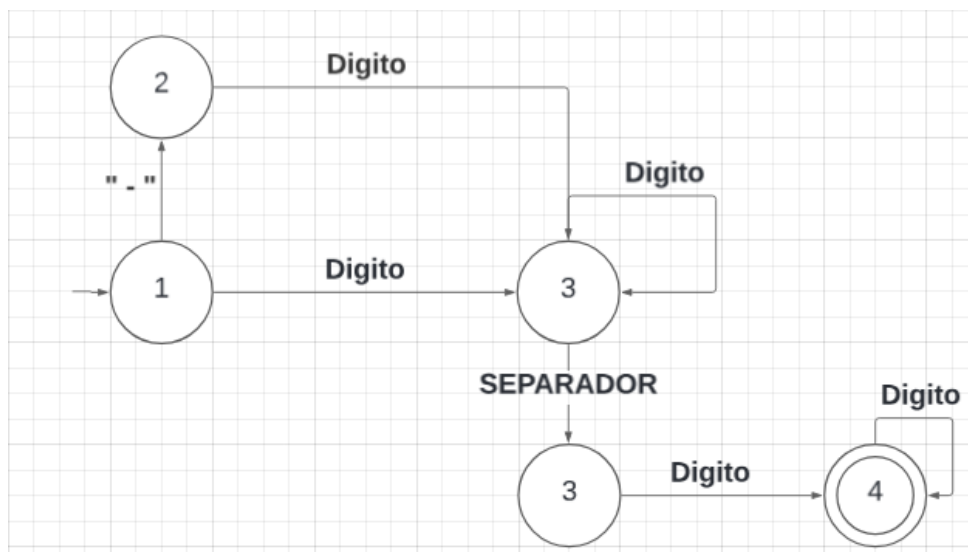


Figura 3 - Autômato da expressão regular Real

- Operação;

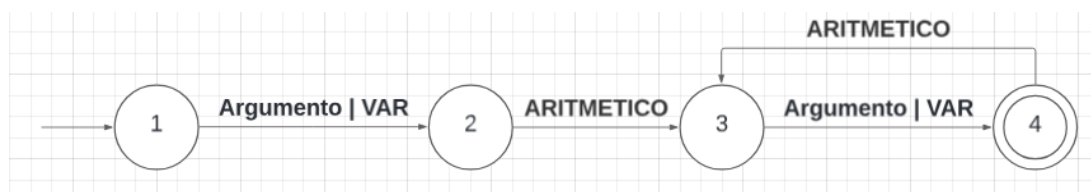


Figura 4 - Autômato da expressão regular Operação

- Atribuição;

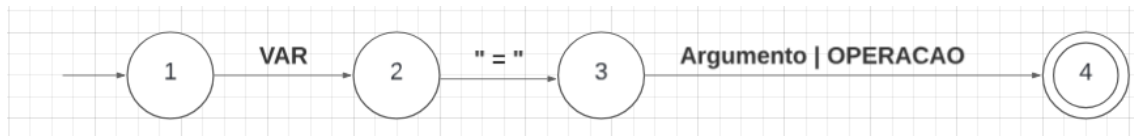


Figura 5 - Autômato da expressão regular Atribuição

- Entrada;

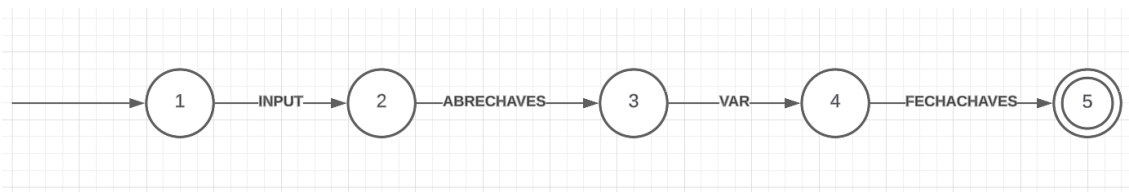


Figura 6 - Autômato da expressão regular Entrada

- Saída;

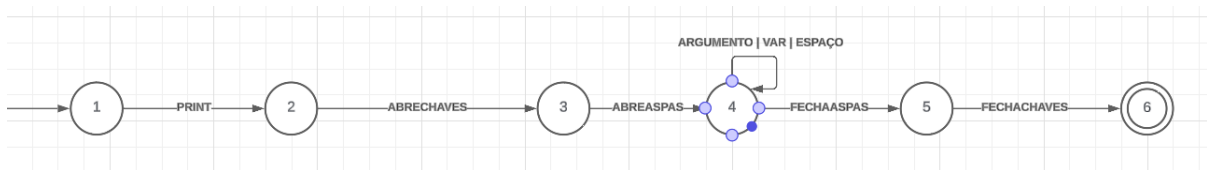


Figura 7 - Autômato da expressão regular Saída

- Condição;

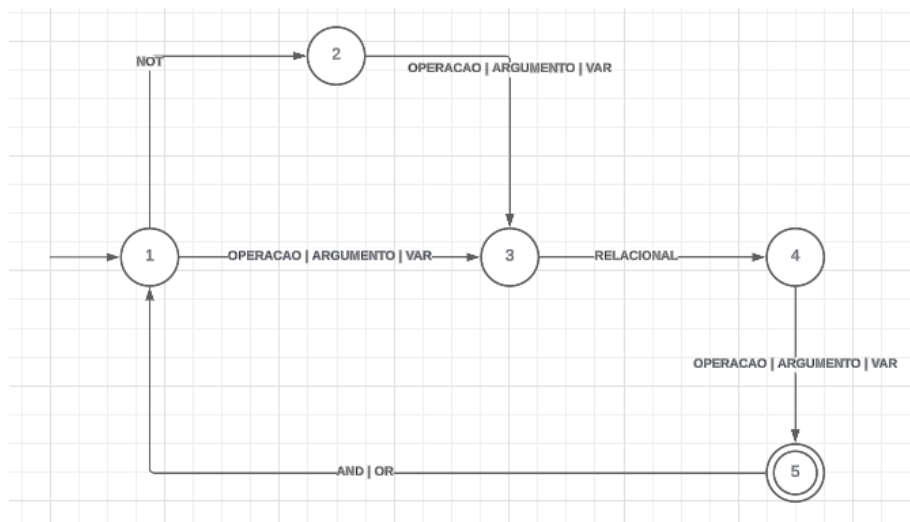


Figura 8 - Autômato da expressão regular Condição

- If;

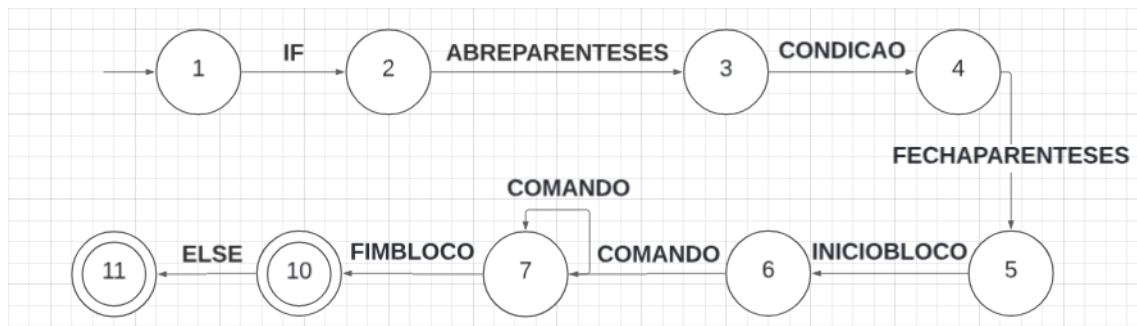


Figura 9 - Autômato da expressão regular If

- Else;

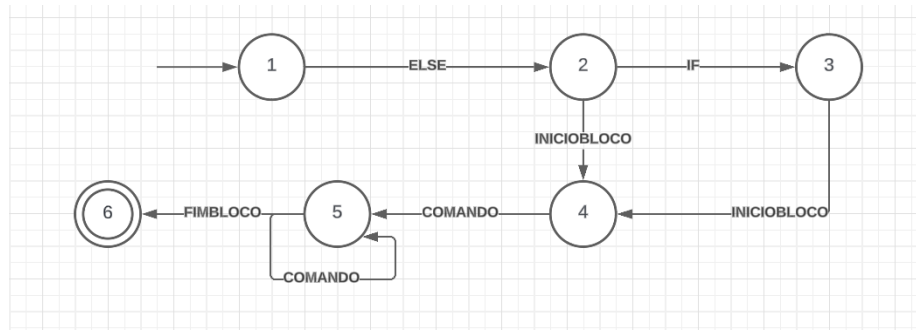


Figura 10 - Autômato da expressão regular Else

- For;

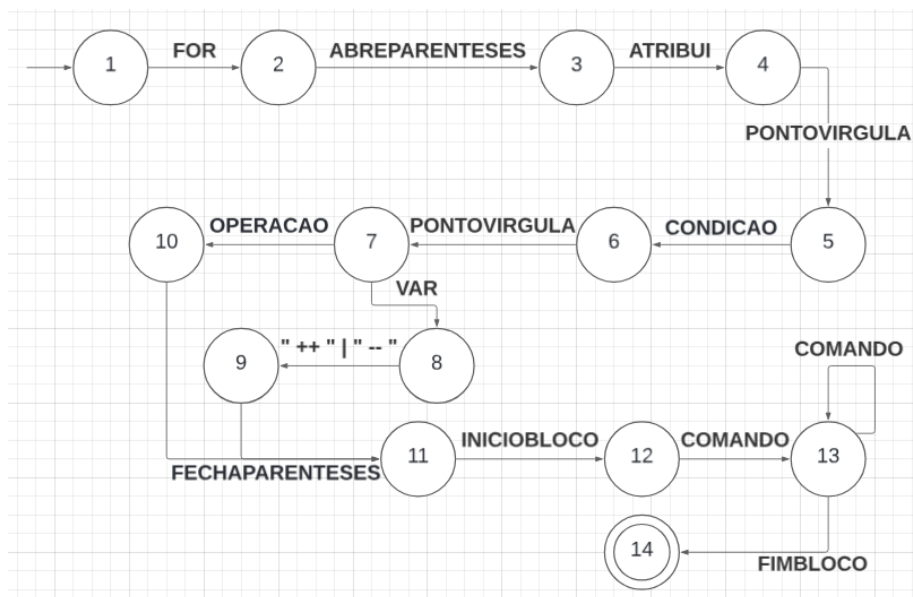


Figura 11 - Autômato da expressão regular For

- While;

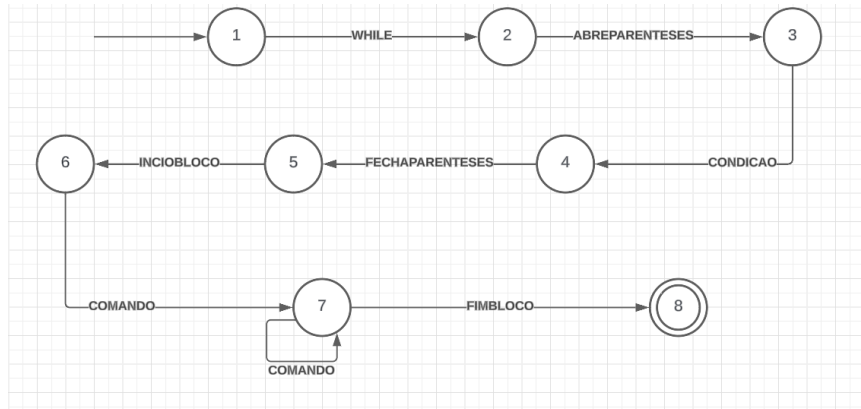


Figura 12 - Autômato da expressão regular While

- Número;

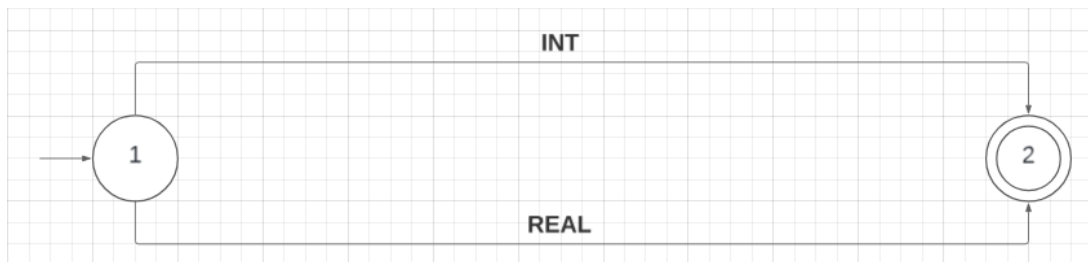


Figura 13 - Autômato da expressão regular Número

- Caractere;



Figura 14 - Autômato da expressão regular Caractere

- Comando.

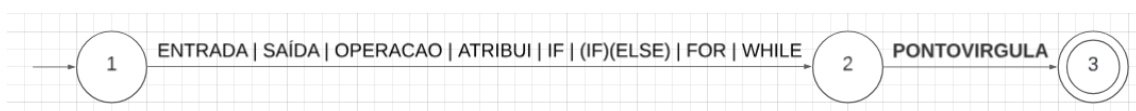


Figura 15 - Autômato da expressão regular Comando